

HTTP Based KV Server

Jayesh Kandpal [[GitHub link](#)]

1 Server Architecture

This section covers the architectural details of the project. The system has two tiers:

1. Frontend: The HTTP Server with an in-memory cache implemented in C++ using the [cpp-httpplib](#) library.
2. Backend: PostgreSQL as the on-disk database.

The server implementation supports 3 types of requests: GET, POST and DELETE. It uses `std::unordered_map<string, string>` as a cache and FCFS as its replacement policy.

GET and POST emulate two different types of workloads. GET emulates a CPU-bound workload, given most of the keys requested by the client are found in the cache. POST emulates a disk-bound workload, since every key-value pair sent by the client needs to be written to the database on disk. DELETE is a mix of both CPU-bound and disk-bound because the key needs to be deleted from the database as well as the cache.

The HTTP server by default uses a threadpool implementation with 8 threads. This implies, there is one main thread which accepts connections and other 8 worker threads which handle the request processing. This can be configured during compilation of the server program by adding the flag `-DCPPHTTPLIB_THREAD_POOL_COUNT=<>` and giving the number of threads in place of `<>`.

1.1 Database details

In the current architecture, we create a new connection to the database every time a server thread needs to access it. So every client request which needs to go to the database will have to create a new connection.

The table where the key-value pairs are stored in the database has a simple schema with 2 columns:

Column	Type	Collation	Nullable	Default
key	character varying(255)		not null	
value	text			

Indexes: "mytable_pkey" PRIMARY KEY, btree (key)

1.2 Cache Details

The in-memory cache is a hash table (`std::unordered_map`). For this, the server maintains an `unordered_map` of 500 key-value pairs. If the size of the cache becomes greater than 500, FCFS is used as an eviction policy. Since cache is a shared structure among all server threads, a mutex lock is used whenever any thread wants to access the cache. Initially, at server startup the cache is empty. Whenever a read (GET) request is received by the client, on cache-miss the required key-value pair is brought from the database into the cache. The complete GET request flow is shown below in Figure 1.

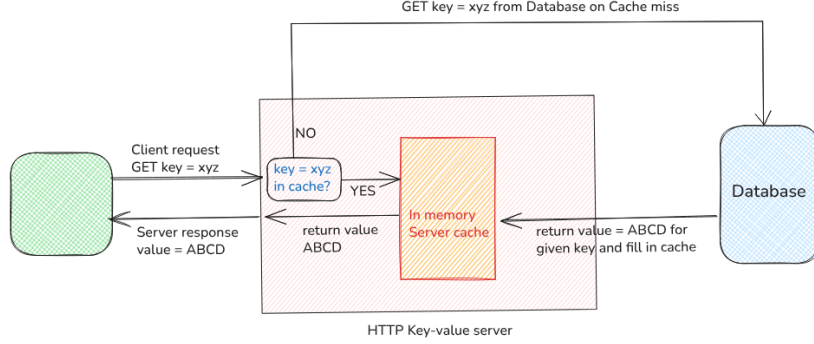


Figure 1: Read request flow. The green box represents the client.

Similarly, the flow for a POST request is such that, the server first inserts the key-value pair into the database. Upon successful insertion into the database, the same key-value pair is then inserted into the cache. If the insertion fails, the server returns a 500 status code.

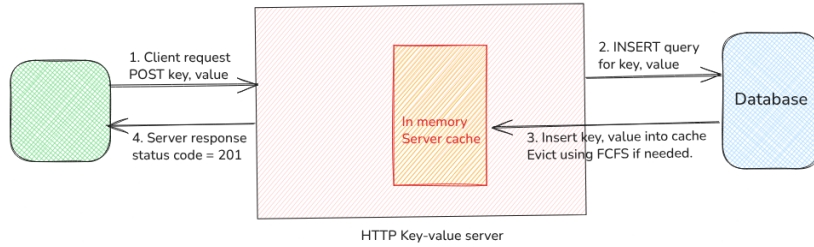


Figure 2: Write request flow. The green box represents the client.

2 Load Generator

The client side is emulated by 2 load generator programs - `read_client.cpp` and `write_client.cpp`. These 2 programs target different resources of the server. `read_client.cpp` sends repeated GET requests for keys which are present in the cache. In this case, the CPU is not waiting for anything and is constantly busy processing the incoming requests. Hence, this program generates CPU-bound workload.

`write_client.cpp` sends repeated POST requests to create new key-value pairs. In this case, the server connects to the on-disk database (PostgreSQL process) and runs an INSERT query. Since, most of the time is spent on waiting for the disk write to complete by the PostgreSQL process, this program generates I/O bound workload.

2.1 Preliminary results

The load-generation setup is such that we pin all 3 processes to different cores of the (8-core) machine. PostgreSQL is pinned to the first 3 cores, Load generator program is pinned to the next 3 cores and the remaining 2 cores are used for the server.

We can observe when performing load-testing for CPU-bound workload with the `read_client.cpp` program with 8 threads for 5 minutes. The average throughput (calculated at client side) is 10135 requests per second and the 2 cores dedicated to the server are constantly at 90% or more utilization. Similarly, when load-testing for I/O-bound workload with `write_client.cpp` program with 8 threads for 5 minutes, as shown by the command `iostat -x 1` the server hits a disk utilization of around 80% and the average throughput comes out to be 67 requests per second. Since in the server implementation, the server makes a new connection to the database for every write, this adds additional overhead and latency. Connection pooling can improve this.